Assignment 1: Dynamic Warehouse Robot Navigation using Reinforcement Learning

Course: Deep Reinforcement Learning (MTech)
Total Marks: 100
Submission Deadline: _____

Problem Statement

A warehouse robot must learn to navigate a **10x10 grid warehouse** to pick up packages from random locations and deliver them to a dispatch zone, while avoiding **moving obstacles** (other robots/forklifts) that follow stochastic movement patterns.

You are required to model this as a Markov Decision Process (MDP), implement the environment, and train agents using **Tabular Q-Learning** and **Deep Q-Network (DQN)**. You will then compare both approaches and provide a critical analysis.

Environment Specification

Parameter	Value
Grid size	10 x 10
Robot start	(0, 0)
Dispatch zone	(9, 9)
Packages	3, randomly placed per episode
Obstacles	2, moving stochastically
Max time steps	200 per episode

State Space

The state at any time step is defined by:

- Robot position: `(x, y)` where `x, y in {0, 1, ..., 9}`
- Package pickup status: a binary vector `[p1, p2, p3]` where `pi = 1` if package `i` has been picked up
- Obstacle positions: `[(ox1, oy1), (ox2, oy2)]`

Action Space

The agent can choose from 6 discrete actions:

Action ID	Description
0	Move Up
1	Move Down
2	Move Left
3	Move Right
4	Pick Up Package
5	Drop / Deliver

Reward Structure

Event	Reward
Successful package pickup	+10
Delivery with all 3 packages collected	+50
Partial delivery (per package)	+15
Collision with a moving obstacle	-20
Wall collision (invalid move)	-5
Pickup action with no package present	-3
Each time step (living penalty)	-1

Stochastic Obstacle Model

At each time step, each obstacle independently:

- Moves with probability `0.7`
- Remains stationary with probability `0.3`

When an obstacle moves, it selects one of the four cardinal directions (up, down, left, right) uniformly at random. If the selected move would take the obstacle off the grid, it stays in its current position.

Questions

Part A: Environment Implementation [15 Marks]

Q1. Implement the warehouse environment as a Python class that follows the Gymnasium (`gym`) API convention. Your implementation must include:

1. An `__init__` method that initializes the grid, places the robot, packages, and obstacles.
2. A `reset()` method that returns `(initial_state, info)` with randomized package and obstacle placements.
3. A `step(action)` method that returns `(next_state, reward, terminated, truncated, info)` and correctly implements:
   - All movement actions with boundary checking
   - Package pickup logic (only succeeds when robot is on a package cell)
   - Delivery logic (only succeeds at the dispatch zone)
   - Obstacle stochastic movement after each agent action
   - Collision detection between the robot and obstacles
   - Episode termination when all packages are delivered or max steps are reached
4. A `render()` method that prints an ASCII representation of the grid showing the robot (`R`), packages (`P`), obstacles (`X`), dispatch zone (`D`), and empty cells (`.`).
5. A `_encode_state()` helper that flattens the full state into a NumPy array suitable for agent input.

Use the following class skeleton as your starting point:

```python
import numpy as np

class WarehouseEnv:
    def __init__(self, grid_size=10, num_packages=3, num_obstacles=2):
        self.grid_size = grid_size
        self.num_packages = num_packages
        self.num_obstacles = num_obstacles
        self.action_space_n = 6
        # Initialize state variables

    def _encode_state(self) -> np.ndarray:
        """Flatten state into a numpy array for the agent."""
        raise NotImplementedError

    def reset(self):
        """Reset environment and return (initial_state, info)."""
        raise NotImplementedError

    def step(self, action):
        """Execute action and return (next_state, reward, terminated, truncated, info).
        raise NotImplementedError

    def _move_obstacles(self):
        """Move each obstacle stochastically per the defined model."""
        raise NotImplementedError

    def render(self):
        """Print ASCII grid visualization."""
        raise NotImplementedError
```

Verify your environment by running 100 episodes with random actions and reporting the average reward, average episode length, and the percentage of episodes where at least one package was picked up.

Part B: Tabular Q-Learning [25 Marks]

Q2. Implement a Tabular Q-Learning agent with the following specifications:

- **Exploration strategy:** Epsilon-greedy with epsilon decaying from `1.0` to `0.01` over the course of training

- **Learning rate (alpha):** `0.1`
- **Discount factor (gamma):** `0.99`
- **Training episodes:** `50,000`

Your implementation must address the following:

(a) [5 Marks] **State Discretization:** The full state space is very large. Design and implement a discretization strategy that maps the continuous/high-dimensional state to a manageable set of discrete keys for the Q-table. Clearly explain and justify your design choice in comments or a short write-up. Consider trade-offs between granularity and table size.

(b) [10 Marks] **Agent Implementation:** Complete the following skeleton:

```python
class QLearningAgent:
    def __init__(self, state_bins, n_actions=6, alpha=0.1, gamma=0.99,
                 epsilon_start=1.0, epsilon_end=0.01, epsilon_decay=0.99995):
        self.q_table = {}
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon_start
        self.epsilon_end = epsilon_end
        self.epsilon_decay = epsilon_decay
        self.n_actions = n_actions

    def discretize_state(self, state: np.ndarray) -> tuple:
        """Convert state array to a discrete tuple key for Q-table lookup."""
        raise NotImplementedError

    def select_action(self, state) -> int:
        """Epsilon-greedy action selection."""
        raise NotImplementedError

    def update(self, state, action, reward, next_state, done):
        """Apply the Q-learning update rule:
        Q(s,a) <- Q(s,a) + alpha * [r + gamma * max_a' Q(s',a') - Q(s,a)]
        """
        raise NotImplementedError
```

(c) [10 Marks] **Training and Visualization:** Train the agent and generate the following plots:

1. **Cumulative reward per episode**, smoothed over a 500-episode rolling window
2. **Epsilon decay curve** over all training episodes

3. **Q-value convergence plot**: track the variance of the maximum Q-value across all visited states over the last 1,000 episodes. Show that this variance decreases over time.

Report the final average reward over the last 1,000 episodes.

Part C: Deep Q-Network [30 Marks]

Q3. Implement a Deep Q-Network (DQN) agent with the following specifications:

- **Experience replay buffer** with capacity `100,000`
- **Target network** updated every `1,000` training steps
- **Batch size:** `64`
- **Neural network architecture:** Minimum 2 hidden layers with ReLU activations
- **Loss function:** Huber loss (smooth L1)
- **Optimizer:** Adam with learning rate `1e-3`
- **Training episodes:** At least `10,000` (train until convergence)

(a) [10 Marks] **Core DQN Implementation:** Complete the following skeletons:

```python
import torch
import torch.nn as nn
from collections import deque
import random

class ReplayBuffer:
    def __init__(self, capacity=100000):
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        """Store a transition in the buffer."""
        raise NotImplementedError

    def sample(self, batch_size):
        """Sample a random batch of transitions."""
        raise NotImplementedError

    def __len__(self):
        return len(self.buffer)


class DQNetwork(nn.Module):
    def __init__(self, state_dim, action_dim, hidden_dims=[128, 128]):
        super().__init__()
        # Build the neural network
        raise NotImplementedError

    def forward(self, x):
        raise NotImplementedError


class DQNAgent:
    def __init__(self, state_dim, action_dim, lr=1e-3, gamma=0.99,
                 target_update_freq=1000, batch_size=64):
        self.policy_net = DQNetwork(state_dim, action_dim)
        self.target_net = DQNetwork(state_dim, action_dim)
        self.target_net.load_state_dict(self.policy_net.state_dict())
        # Initialize optimizer, replay buffer, parameters

    def select_action(self, state, epsilon):
        """Epsilon-greedy action selection using the policy network."""
        raise NotImplementedError
```

```python
    def train_step(self, replay_buffer):
        """Sample a batch, compute Huber loss against target network, backpropagate."""
        raise NotImplementedError

    def update_target_network(self):
        """Copy policy network weights to target network."""
        raise NotImplementedError

    def save(self, path):
        """Save the trained model to a .pth file."""
        raise NotImplementedError
```

(b) [10 Marks] **DQN Extension:** Implement **one** of the following extensions. State clearly which extension you chose and provide a brief justification (3-5 sentences) for why it is appropriate for this problem.

- **Double DQN** (Van Hasselt et al., 2016): Use the policy network to select actions and the target network to evaluate them, reducing overestimation bias.
- **Dueling DQN** (Wang et al., 2016): Modify the network architecture to have separate streams for state-value `V(s)` and advantage `A(s, a)`, combined as `Q(s, a) = V(s) + A(s, a) - mean(A)`.
- **Prioritized Experience Replay** (Schaul et al., 2016): Replace uniform sampling in the replay buffer with priority-based sampling proportional to TD error magnitude. Include importance sampling weights to correct the bias.

(c) [10 Marks] **Training, Evaluation, and Comparison:** Train the DQN agent and produce:

1. **Training loss curve** (averaged over every 100 training steps)
2. **Reward per episode** (smoothed over a 200-episode window)
3. **Comparative bar chart or line plot**: Final performance (average reward over last 500 episodes) of Tabular Q-Learning vs. base DQN vs. your chosen DQN extension

Discuss which agent performs best and provide reasoning.

Part D: Analysis and Report [30 Marks]

Q4. Perform the following analyses and present your findings in a structured report (PDF, max 10 pages, IEEE double-column format).

(a) [10 Marks] **Ablation Study:**

Run experiments varying the following hyperparameters and present results in a table:

Experiment	Values to test
Discount factor (gamma)	0.8, 0.9, 0.99
Replay buffer size	1,000 / 10,000 / 100,000
Target update frequency	100 / 1,000 / 5,000

For each variation, report:

- Average reward over last 500 episodes
- Number of episodes to reach a reward threshold of +40

Discuss which hyperparameters have the most significant impact and explain why.

(b) [10 Marks] **Policy Visualization:**

For a **fixed obstacle configuration** (set a specific random seed):

1. Generate a **heatmap** of the learned value function `V(s) = max_a Q(s, a)` across all grid cells (with all packages not yet picked up).
2. Overlay **policy arrows** on the grid showing the best action at each cell.
3. Render at least **3 complete episode trajectories** of the trained DQN agent on the grid, showing the path taken, packages collected, and final outcome.

Use `matplotlib` for all visualizations.

(c) [10 Marks] **Critical Analysis:**

Answer the following questions with technical depth. Support your answers with evidence from your experiments.

1. **State space explosion:** Calculate the exact size of the full state space for this problem. Explain why tabular Q-learning struggles and how function approximation (DQN) addresses this.
2. **Stochastic obstacles:** Compare the learning curves of your DQN agent trained with stochastic obstacles against a variant trained with **static obstacles** (obstacles fixed at their initial positions). How does stochasticity affect convergence speed and final performance?
3. **Sim-to-real gap:** If this policy were to be deployed on a physical warehouse robot, what challenges would arise? Discuss at least three specific issues (e.g., sensor noise, continuous

action space, partial observability).

4. **Beyond DQN:** Propose one algorithm beyond DQN (e.g., PPO, SAC, hierarchical RL, model-based RL) that could improve performance in this setting. Explain why it would be a better fit and what changes to the code would be needed.

Submission Requirements

Item	Format / Details
Source code	`.py` files, runnable with `python main.py`
Report	PDF, max 10 pages, IEEE double-column format
Plots and figures	Embedded in report; raw data also submitted as `.csv`
Trained model weights	Saved `.pth` file(s) for DQN
Dependencies	`requirements.txt` listing all packages used
README	Brief instructions on how to reproduce your results

Evaluation Criteria

Criterion	Marks
Correct environment dynamics and reward logic	15
Working Q-Learning agent with convergence evidence	25
Working DQN agent with extension	30
Analysis depth, visualizations, and report quality	20
Code quality, modularity, and documentation	10
Total	**100**

References

1. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018. (Chapters 6, 16)
2. V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529-533, 2015.
3. H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.
4. Z. Wang et al., "Dueling network architectures for deep reinforcement learning," *Proceedings of the International Conference on Machine Learning (ICML)*, 2016.
5. T. Schaul et al., "Prioritized experience replay," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.

Note: You may use standard Python libraries including `numpy`, `torch`, `matplotlib`, `gymnasium`, and `collections`. Use of pre-built RL libraries such as Stable-Baselines3 is **not permitted** for the core implementation (Parts A-C). You may use them only for validation purposes in Part D.